

```
In[1]:= Print["Wolfram Language ready!"]  
Print[$Version]  
Wolfram Language ready!  
14.3.0 for Linux x86 (64-bit) (July 8, 2025)
```

Assignment 04: Exploring Spiking Neurons with Wolfram

Eduardo Cabrera-Lopez

ITAI 4374 – Neuroscience as a Model for AI

Houston Community College

Spring 2026

Part A – Solving the LIF Equation

```
In[18]:= (* ===== Neuron Parameters ===== *)
```

```
tau = 0.020;
Vrest = -70.0;
Vthresh = -55.0;
Vreset = -75.0;
R = 1.0;
```

```
(* ===== Input Current ===== *)
```

```
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];
```

```
Print["Parameters defined."]
```

```
Parameters defined.
```

Check Your Knowledge

1. An If statement is used to define the input current. In simple terms, it only lets the current flow within a particular time period. The condition is false at $t = 0.03$, therefore the function returns 0.0 and the neuron doesn't get any current. The condition is true at $t = 0.2$, therefore the function returns 18.0 and the neuron gets input current. At $t = 0.5$, the condition is false again, which means the current has stopped.
2. The `:=` operator is employed because it checks the function every time it is invoked. This lets the current change based on the value of time. If we used `=` instead, the expression would only be checked once when it is defined, and it would not change correctly for different time values.

```
In[1]:= ClearAll["Global`*"]
```

```
tau = 0.020;
Vrest = -70.0;
Vthresh = -55.0;
Vreset = -75.0;
R = 1.0;
```

```
Iinput[t_] := If[0.05 ≤ t ≤ 0.4, 18.0, 0.0];
```

```
result = Reap[
  sol = NDSolveValue[
    {
      tau*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
      V[0] == Vrest,
      WhenEvent[V[t] ≥ Vthresh,
        {
          Sow[t],
          V[t] → Vreset
        }
      ]
    },
    V,
    {t, 0, 0.5}
  ]
];
```

```
spikeTimes = If[Length[result[[2]]] > 0, result[[2, 1]], {}];
spikeCount = Length[spikeTimes];
```

```
Print["Neuron fired ", spikeCount, " spikes"]
Print["Spike times: ", spikeTimes]
```

```
Neuron fired 8 spikes
```

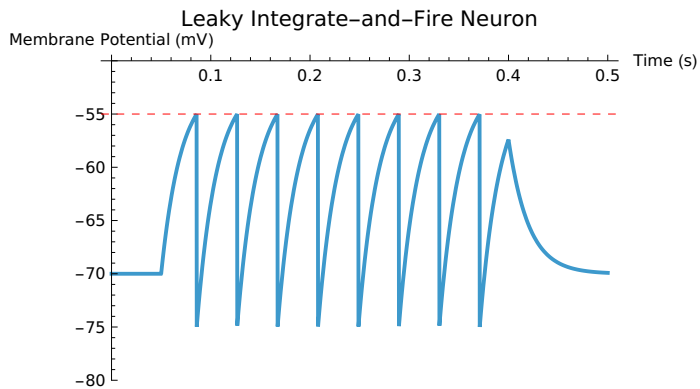
```
Spike times:
{0.0858352, 0.126573, 0.16731, 0.208048, 0.248786, 0.289523, 0.330261, 0.370999}
```

```

In[13]:= Plot[
  sol[t],
  {t, 0, 0.5},
  PlotRange → {-80, -50},
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → "Leaky Integrate-and-Fire Neuron",
  Epilog → {Red, Dashed, InfiniteLine[{{0, Vthresh}, {1, Vthresh}}]}
]

```

Out[13]=



Check Your Knowledge

1. Instead of a straight line, the voltage between spikes becomes a curve. This arises because the LIF model has a leak term that keeps pulling the voltage back to the resting potential. The membrane potential rises in a curved pattern until it hits the threshold.
2. After $t > 0.4$ seconds, when the current stops off, the voltage progressively moves back to the resting potential. This happens because the leak term in the LIF equation is stronger when there is no input current, which makes the neuron relax back to its resting state.

Part B – Interactive LIF Dashboard

```

In[14]:= Manipulate[
  Module[{result, solLocal, st, sc, iFunc},

    iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

    result = Reap[
      solLocal = NDSolveValue[
        {
          tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
          V[0] == Vrest,
          WhenEvent[V[t] ≥ thresh,
            {
              Sow[t],
              V[t] → Vreset
            }
          ]
        },
        V,
        {t, 0, 0.5}
      ]
    ];

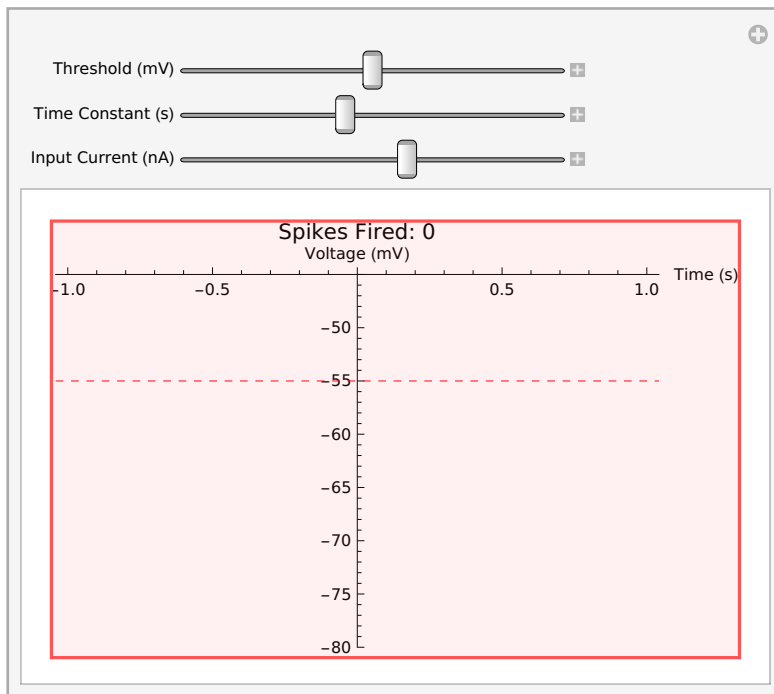
    st = If[Length[result[[2]] > 0, result[[2, 1]], 0];
    sc = Length[st];

    Plot[
      solLocal[t],
      {t, 0, 0.5},
      PlotRange → {-80, -45},
      AxesLabel → {"Time (s)", "Voltage (mV)"},
      PlotLabel → StringForm["Spikes Fired: ``", sc],
      Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]}
    ],

    {{thresh, -55.0, "Threshold (mV)"}, -65, -45, 1},
    {{tauVal, 0.020, "Time Constant (s)"}, 0.005, 0.100, 0.005},
    {{currentVal, 18.0, "Input Current (nA)"}, 0, 40, 1}
  ]

```

Out[14]=




```

In[15]:= currentVal = 25;
thresh = -55;
tauVal = 0.020;

iFunc[t_] := If[0.05 ≤ t ≤ 0.4, currentVal, 0.0];

resultB = Reap[
  solB = NDSolveValue[
    {
      tauVal * V'[t] == -(V[t] - Vrest) + R * iFunc[t],
      V[0] == Vrest,
      WhenEvent[V[t] ≥ thresh, { Sow[t], V[t] → Vreset }]
    },
    V,
    {t, 0, 0.5}
  ]
];

stB = If[Length[resultB[[2]]] > 0, resultB[[2, 1]], {}];
scB = Length[stB];

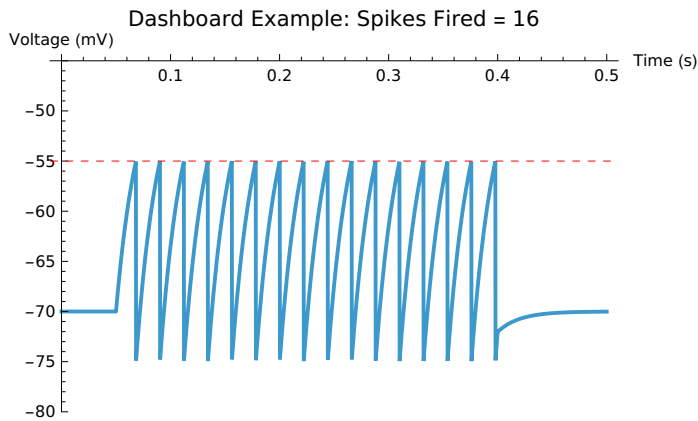
Print["Spikes Fired: ", scB];

Plot[
  solB[t],
  {t, 0, 0.5},
  PlotRange → {-80, -45},
  AxesLabel → {"Time (s)", "Voltage (mV)"},
  PlotLabel → StringForm["Dashboard Example: Spikes Fired = ``", scB],
  Epilog → {Red, Dashed, InfiniteLine[{{0, thresh}, {1, thresh}}]}
]

Spikes Fired: 16

```

Out[23]=



Knowledge Check

1. I discovered that the neuron started firing at roughly 17 nA when I used the dashboard with the default tau and threshold parameters. This is the rheobase, which is the smallest amount of current that can make a neuron fire at least one spike.
2. The cell ceased firing when I set the current to 25 nA and slowly raised the threshold from -65 mV to -45 mV. This happened because the membrane potential had to reach a higher level for a spike to arise.
3. When I set the threshold to -55 and the current to 20, the neuron with $\tau = 0.010$ made more spikes than the one with $\tau = 0.050$. The neuron with $\tau = 0.050$ had a smoother curve for its membrane potential. This makes sense because a smaller time constant allows the neuron respond more quickly, whereas a larger one makes the voltage change more slowly.

Reflection

The Manipulate dashboard let me figure out how changing neuron parameters affects spiking activity. I could quickly see how changing the threshold, time constant, or input current influenced the amount of spikes by moving the sliders. One thing that jumped out was how easily the neuron reacted to the threshold value. Even tiny modifications in the threshold made the cell fire a lot more often. It was much easier to understand how neurons combine input signals when I could see the spikes change in real time.

Part C – Firing Rate Curve (f-I Curve)

```

In[24]:= fiData = Table[
  Module[{resultFI, solFI, stFI, scFI},

    resultFI = Reap[
      solFI = NDSolveValue[
        {
          tau*V'[t] == -(V[t] - Vrest) + R*Icurr,
          V[0] == Vrest,
          WhenEvent[V[t] ≥ Vthresh,
            {
              Sow[t],
              V[t] → Vreset
            }
          ]
        },
        V,
        {t, 0, 0.5}
      ]
    ];

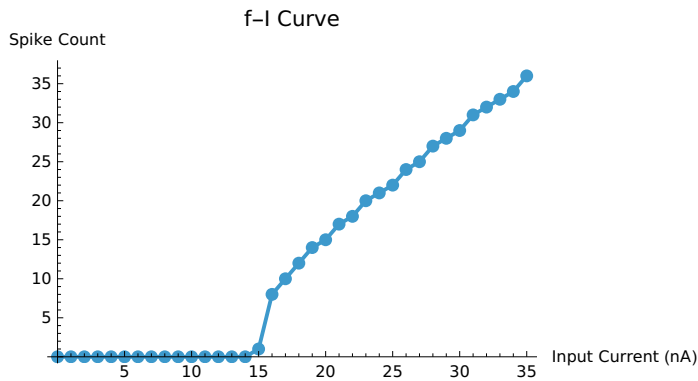
    stFI = If[Length[resultFI[[2]]] > 0, resultFI[[2, 1]], {}];
    scFI = Length[stFI];

    {Icurr, scFI}
  ],
  {Icurr, 0, 35, 1}
];

ListLinePlot[
  fiData,
  AxesLabel → {"Input Current (nA)", "Spike Count"},
  PlotLabel → "f-I Curve",
  PlotMarkers → Automatic,
  PlotRange → All
]

```

Out[25]=



Check Your Knowledge

1. According to my f-I curve, the neuron began firing at approximately 17 nA. This is quite similar to the rheobase since it shows the least amount of current needed to make the neuron fire.
2. The f-I curve isn't entirely straight. It starts off flat at low current values since the neuron doesn't fire at all until it reaches a certain level. The curvature goes up after that as the current goes up. If the line is absolutely straight, it means that the neuron encodes the strength of the stimulus in a way that is completely proportionate at all levels of input.
3. There isn't a substantial saturation impact at high currents in my LIF model. As the input current goes up, the spike count keeps going up. This makes sense because the basic LIF model doesn't incorporate many of the biological restrictions that real neurons have, such as adaptation, ion channel exhaustion, or a severe refractory effect.

Part D – Encoding a Signal as Spikes

```
In[26]:= Isine[t_] := 15.0 + 12.0*Sin[2*Pi*5*t];
```

```
resultSine = Reap[
  solSine = NDSolveValue[
    {
      tau*V'[t] == -(V[t] - Vrest) + R*Isine[t],
      V[0] == Vrest,
      WhenEvent[V[t] ≥ Vthresh,
        {
          Sow[t],
          V[t] → Vreset
        }
      ]
    }
  ]
```

```

    }
  ]
},
V,
{t, 0, 1.0}
]
];

```

```

spikeTimesSine = If[Length[resultSine[[2]] > 0, resultSine[[2, 1]], {}];
spikeCountSine = Length[spikeTimesSine];

```

```

Print["Sine-wave spike count: ", spikeCountSine];
Print["Spike times: ", spikeTimesSine];

```

```

plotVoltage = Plot[
  solSine[t],
  {t, 0, 1.0},
  PlotRange → {-80, -50},
  AxesLabel → {"Time (s)", "Membrane Potential (mV)"},
  PlotLabel → "Membrane Potential with Sine-Wave Input"
];

```

```

plotInput = Plot[
  Isine[t],
  {t, 0, 1.0},
  AxesLabel → {"Time (s)", "Input Current (nA)"},
  PlotLabel → "Sine-Wave Input Current"
];

```

```

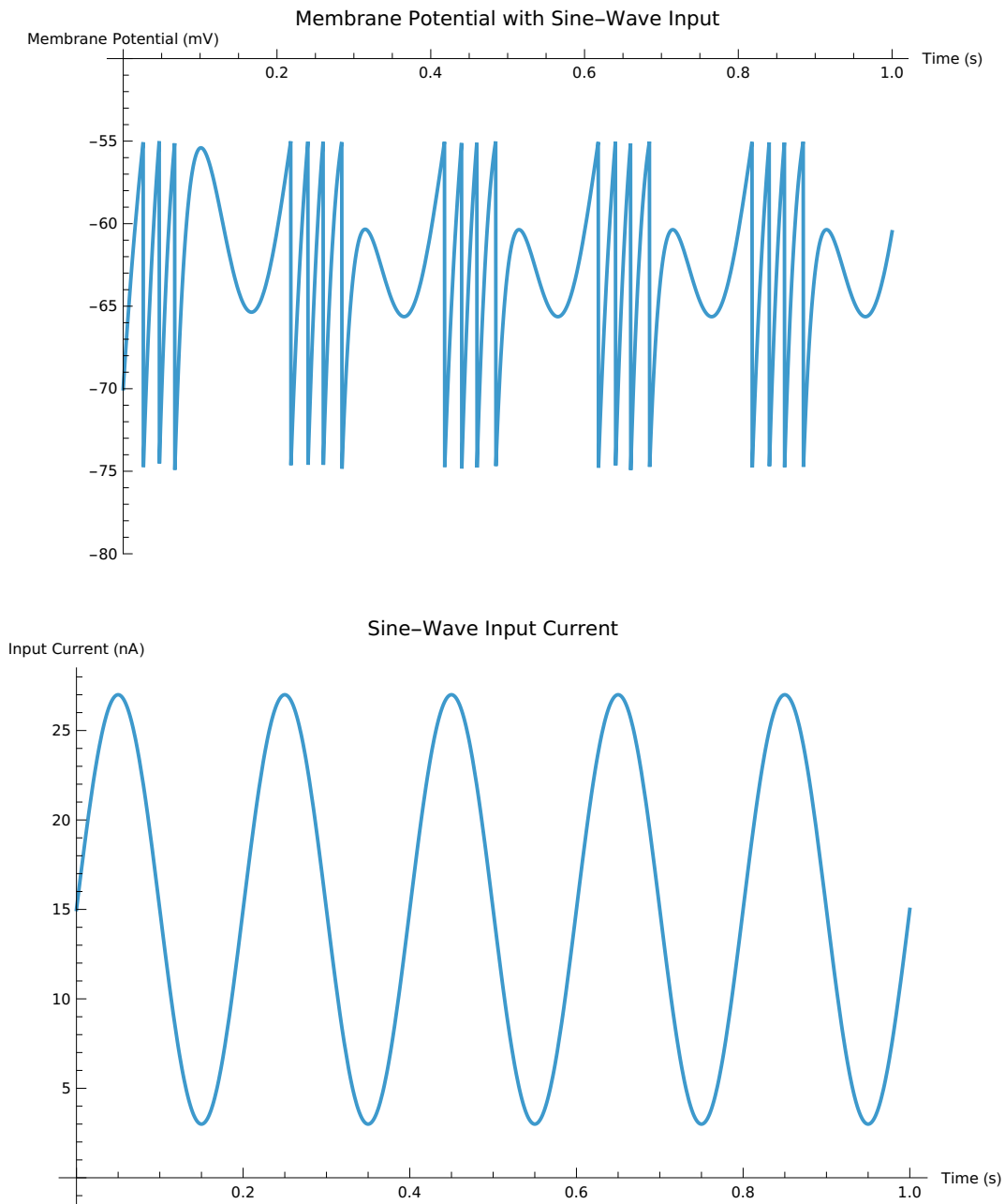
GraphicsColumn[{plotVoltage, plotInput}]

```

Sine-wave spike count: 19

Spike times: {0.0260992, 0.0468183, 0.0671122, 0.217907, 0.240209,
 0.260038, 0.284442, 0.41797, 0.440257, 0.460087, 0.484523, 0.617971,
 0.640257, 0.660087, 0.684524, 0.817971, 0.840257, 0.860087, 0.884524}

Out[34]=



Reflection

When I looked at the spike train and the sine wave input, I saw that the neuron fired more often when the sine wave was close to its peaks and less often when the signal was close to its troughs. In that way, the spike pattern fit the geometry of the signal quite well. The most important factor that was kept was how the input changed over time, since higher sections of the sine wave made the spiking denser. The neuron doesn't directly reproduce the waveform; instead, it turns a continuous input into discrete spikes. This means that the exact smooth shape of the original signal was lost.

Part E – Experiments

```

In[35]:= thresholdData = Table[
  Module[{resultTh, solTh, stTh, scTh},

    resultTh = Reap[
      solTh = NDSolveValue[
        {
          tau * V'[t] == -(V[t] - Vrest) + R * Iinput[t],
          V[0] == Vrest,
          WhenEvent[V[t] ≥ th,
            {
              Sow[t],
              V[t] → Vreset
            }
          ]
        },
        V,
        {t, 0, 0.5}
      ]
    ];

    stTh = If[Length[resultTh[[2]]] > 0, resultTh[[2, 1], 0];
    scTh = Length[stTh];
    {th, scTh}
  ],
  {th, -65, -45, 5}
];

Grid[
  Prepend[thresholdData, {"Threshold (mV)", "Spike Count"}],
  Frame → All
]

ListPlot[

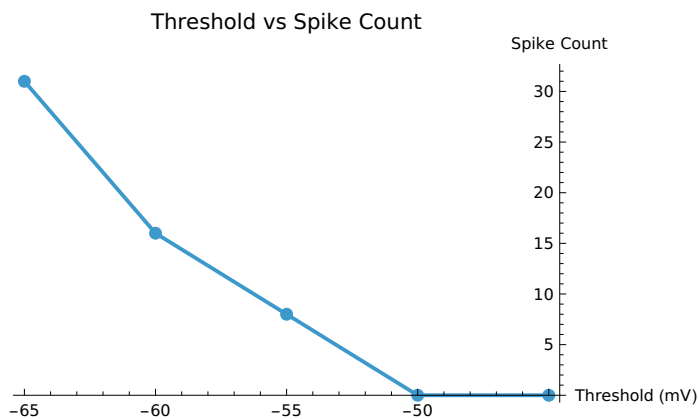
```

```
thresholdData,
Joined → True,
AxesLabel → {"Threshold (mV)", "Spike Count"},
PlotLabel → "Threshold vs Spike Count",
PlotMarkers → Automatic
]
```

Out[36]=

Threshold (mV)	Spike Count
-65	31
-60	16
-55	8
-50	0
-45	0

Out[37]=



Experiment 1: Sweep of the Threshold

1. I reduced the threshold from -65 mV to -45 mV in 5 mV steps for this test.
2. I thought that raising the threshold would cut down on the amount of spikes because the neuron would need more voltage to fire.
3. That's exactly what occurred. Lower thresholds led to increased spiking, while higher thresholds led to fewer spikes and eventually made the neuron stop functioning.
4. This makes sense because the LIF equation forces the voltage go up toward the threshold. Setting the threshold higher makes it harder for the membrane potential to achieve it before the input current stops.


```

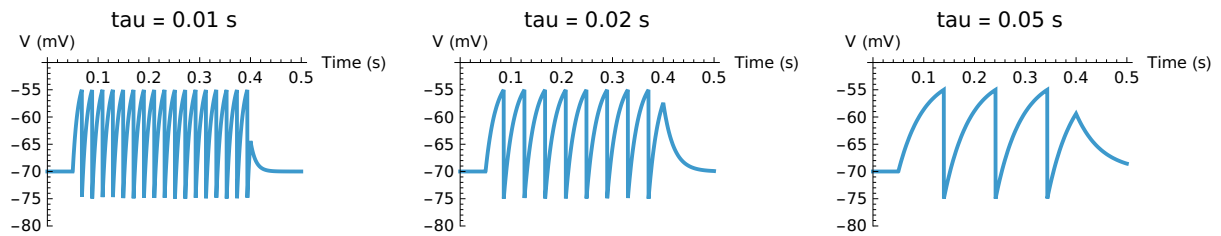
In[38]:= makeTauPlot[tauTest_] := Module[{resultTau, solTau},
  resultTau = Reap[
    solTau = NDSolveValue[
      {
        tauTest*V'[t] == -(V[t] - Vrest) + R*Iinput[t],
        V[0] == Vrest,
        WhenEvent[V[t] ≥ Vthresh,
          {
            Sow[t],
            V[t] → Vreset
          }
        ]
      },
      V,
      {t, 0, 0.5}
    ]
  ];

  Plot[
    solTau[t],
    {t, 0, 0.5},
    PlotRange → {-80, -50},
    AxesLabel → {"Time (s)", "V (mV)"},
    PlotLabel → StringForm["tau = `` s", tauTest]
  ]
];

GraphicsRow[
  {
    makeTauPlot[0.010],
    makeTauPlot[0.020],
    makeTauPlot[0.050]
  }
]

```

Out[39]=



Experiment 2: Comparison of Time Constants

1. I looked at tau values of 0.010, 0.020, and 0.050 seconds for this test.
2. I thought that a smaller time constant would make the neuron react faster and a bigger time constant would make the voltage fluctuate more slowly.
3. That's what I saw. The smaller tau values made the membrane potential curve climb faster and spike more, whereas the bigger tau value had it rise more slowly and smoothly.
4. This makes sense because the membrane time constant controls how rapidly the voltage can fluctuate when the input changes. A short time constant suggests that the neuron reacts rapidly, while a long time constant means that it takes longer to integrate.

```

In[40]:= makeFI[tauTest_] := Table[
  Module[{resultTemp, solTemp, stTemp, scTemp},

    resultTemp = Reap[
      solTemp = NDSolveValue[
        {
          tauTest*V'[t] == -(V[t] - Vrest) + R*Icurr,
          V[0] == Vrest,
          WhenEvent[V[t] ≥ Vthresh,
            {
              Sow[t],
              V[t] → Vreset
            }
          ]
        },
        V,
        {t, 0, 0.5}
      ]
    ];

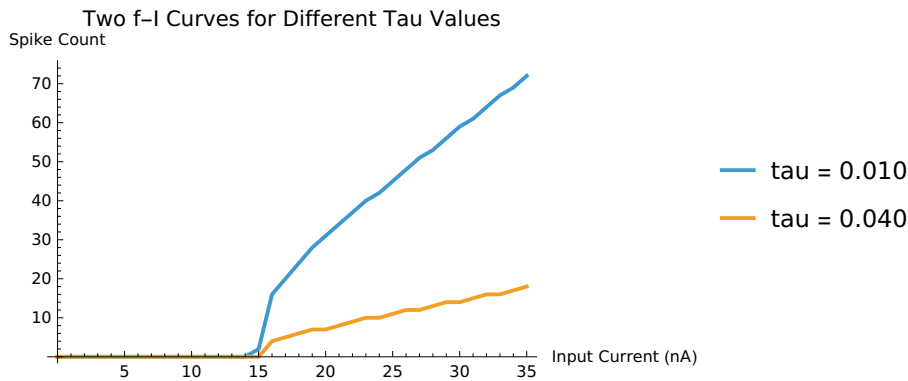
    stTemp = If[Length[resultTemp[[2]]] > 0, resultTemp[[2, 1]], {}];
    scTemp = Length[stTemp];
    {Icurr, scTemp}
  ],
  {Icurr, 0, 35, 1}
];

fiTau1 = makeFI[0.010];
fiTau2 = makeFI[0.040];

ListLinePlot[
  {fiTau1, fiTau2},
  AxesLabel → {"Input Current (nA)", "Spike Count"},
  PlotLabel → "Two f-I Curves for Different Tau Values",
  PlotLegends → {"tau = 0.010", "tau = 0.040"},
  PlotRange → All
]

```

Out[43]=



Experiment 3: Two Distinct f-I Curves

1. I made two f-I curves for this experiment, one with $\tau = 0.010$ and the other with $\tau = 0.040$.
2. I thought that the neuron with the smaller τ would be more sensitive and fire more spikes when the same input current was applied.
3. The results backed that up. The curve for $\tau = 0.010$ went up faster, but the curve for $\tau = 0.040$ stayed lower over most of the present range.
4. This is logical, as a reduced time constant facilitates a more rapid alteration of the membrane voltage, hence increasing the neuron's sensitivity to input current.

Part F – Final Reflection

1. As I slowly raised the input current from below the threshold to over it, the neuron moved from not firing at all to firing many times. When the current was low, the voltage would go up but never exceed the limit. The neuron began to emit spikes at regular intervals when the input current was strong enough. This threshold behavior indicates that biological neurons don't react the same way to every input. Instead, they work more like decision-making units that only fire when the input is strong enough.
2. The f-I curve I had wasn't precisely straight. At first, it stayed flat, but after the current reached the firing threshold, it started to climb. This is essential for biology because it indicates that neurons encode stimulus strength by firing rate, but only once the input is powerful enough to matter. In other words, weak inputs might not be taken into account, while stronger inputs are shown by more frequent spiking.
3. Changing the temporal constant of the membrane had a big effect on how it fired. The neuron responded faster and typically made more spikes when the time constants were less. When the time

constants were larger, the voltage changed more slowly and the curves were smoother. In biological terms, neurons with short time constants may be better at picking up signals that change quickly, whereas neurons with longer time constants may be better at putting information together over time.

4. The LIF model does a good job of capturing certain crucial aspects of real neurons, such as how they leak through their membranes, how they fire based on a threshold, and how they reset after a spike. But it also misses a lot of critical biological information. It doesn't talk about the form of dendrites, the effects of neurotransmitters, adaptability, or the complete complexity of how action potentials are made. So, it's a useful simplified model, but it doesn't show how real neurons work in full.

5. Wolfram's Manipulate[] function made the assignment a lot easier to understand than merely reading about neurons in theory. It helped me relate the arithmetic to the way the neuron acted when I could tweak the sliders and observe the spike pattern change right away. The most shocking thing I learned was how easily the neuron changed when the threshold changed. A slight change in the threshold could make a big difference in the number of spikes.